

Ερώτημα 1:

Η συνάρτηση αξιολόγησης ελέγχει αν, με τη κίνηση action, το pacman βρεθεί στο επόμενο state στην ίδια θέση με ένα ghost. Αν ναι, τότε επιστρέφει τη μικρότερη δυνατή τιμή του συστήματος για να μην διαλέξει αυτή τη κίνηση σε καμία περίπτωση η συνάρτηση getAction. Τη μικρότερη δυνατή τιμή του συστήματος την επιστρέφει επίσης αν η κίνηση action είναι ίση με STOP γιατί δεν θέλουμε να χάσει χρόνο το pacman δίνοντας την ευκαιρία σε ένα ghost να το πλησιάσει. Αν δεν συμβαίνει κάτι από τα παραπάνω, τότε υπολογίζουμε τη manhattan απόσταση της νέας θέσης του pacman με τη θέση του κάθε φαγητού στο τωρινό state. Αν μια απόσταση είναι 0 τότε επιστρέφουμε τη μεγαλύτερη δυνατή τιμή του συστήματος για να επιλέξει τη κίνηση action ώστε να φάει το φαγητό. Τις τοποθετούμε όλες σε μία λίστα all_manhattans αφού τις έχουμε αντιστρέψει ώστε η μικρότερη manhattan απόσταση να έχει τη μεγαλύτερη τιμή ανάμεσά τους και να επιλέξει εκείνη η συνάρτηση getAction. Το μεγαλύτερο στοιχείο της λίστας είναι η μικρότερη Manhattan απόσταση οπότε επιστρέφουμε το max στοιχείο της λίστας all_manhattans.

Ερώτημα 2:

Ο κώδικας βασίζεται στον αλγόριθμο minimax των διαφανειών 19 και 20 της ενότητας 6. Το terminal-test είναι αν το βάθος που βρισκόμαστε στην αναζήτηση είναι ίσο με το βάθος του δένδρου παιχνιδιού ή αν στο τωρινό state κερδίζει το pacman ή αν στο τωρινό state χάνει το pacman. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Η συνάρτηση max_value είναι ακριβώς όπως στις προαναφερθείσες διαφάνειες. Για τη min_value, το terminal-test είναι αν στο τωρινό state κερδίζει το pacman ή αν στο τωρινό state χάνει το pacman. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Ακολουθούμε πάλι τις διαφάνειες με μία διαφορά: Αν ψάχνουμε για το τελευταίο ghost, τότε μετά καλούμε τη max_value γιατί παίζει ο παίκτης max(δηλαδή το pacman). Αν όμως δεν ψάχνουμε για το τελευταίο ghost, τότε καλούμε τη min_value για το επόμενο ghost(δηλαδή για το ghost με index μεγαλύτερο κατά 1 από το index του προηγούμενου ghost. Για κάθε παιδί του state-ρίζα, που είναι το state που παράγεται με κάθε κίνηση action που μπορεί να κάνει το pacman, καλούμε τη min_value και βρίσκουμε τη minimax τιμή του. Έπειτα, βάζουμε τη πλειάδα (minimax_value, action) σε μία λίστα. Επειδή ο κόμβος-ρίζα είναι max, επιστρέφουμε τη κίνηση που έχει τη μεγαλύτερη minimax τιμή.

Ερώτημα 3:

Ο κώδικας βασίζεται στον αλγόριθμο άλφα-βήτα των διαφανειών 48 και 49 της ενότητας 6. Το terminal-test είναι αν το βάθος που βρισκόμαστε στην αναζήτηση είναι ίσο με το βάθος του δένδρου παιχνιδιού ή αν στο τωρινό state κερδίζει το pacman ή αν στο τωρινό state χάνει το pacman. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Η συνάρτηση max_value είναι ακριβώς όπως στις

προαναφερθείσες διαφάνειες. Για τη `min_value`, το `terminal-test` είναι αν στο τωρινό state κερδίζει το `pacman` ή αν στο τωρινό state χάνει το `pacman`. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Ακολουθούμε πάλι τις διαφάνειες με μία διαφορά: Αν ψάχνουμε για το τελευταίο ghost, τότε μετά καλούμε τη `max_value` γιατί παίζει ο παίκτης `max`(δηλαδή το `pacman`). Αν όμως δεν ψάχνουμε για το τελευταίο ghost, τότε καλούμε τη `min_value` για το επόμενο ghost(δηλαδή για το ghost με index μεγαλύτερο κατά 1 από το index του προηγούμενου ghost. Για κάθε παιδί του state-ρίζα, που είναι το state που παράγεται με κάθε κίνηση `action` που μπορεί να κάνει το `pacman`, καλούμε τη `min_value` και βρίσκουμε την `alpha` τιμή του. Πρέπει όμως να ανανεώνουμε και τη μεταβλητή `a` για τον κόμβο-ρίζα που περιέχει τη μεγαλύτερη τιμή που έχουμε βρει μέχρι στιγμής. Έπειτα, βάζουμε τη πλειάδα (`alpha_value, action`) σε μία λίστα. Επειδή ο κόμβος-ρίζα είναι `max`, επιστρέφουμε τη κίνηση που έχει τη μεγαλύτερη `alpha` τιμή.

Ερώτημα 4:

Το `terminal-test` είναι αν το βάθος που βρισκόμαστε στην αναζήτηση είναι ίσο με το βάθος του δένδρου παιχνιδιού ή αν στο τωρινό state κερδίζει το `pacman` ή αν στο τωρινό state χάνει το `pacman`. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Η συνάρτηση `max_value` είναι ακριβώς όπως στα προηγούμενα ερωτήματα. Για την `expectimax_value`, το `terminal-test` είναι αν στο τωρινό state κερδίζει το `pacman` ή αν στο τωρινό state χάνει το `pacman`. Τότε πρόκειται για state-φύλλο του δένδρου παιχνιδιού και επιστρέφουμε την τιμή αξιολόγησης για το state αυτό. Η `expectimax` τιμή ενός κόμβου είναι ίση με το μέσο όρο των τιμών των παιδιών της, δηλαδή (άθροισμα τιμών όλων των παιδιών) / (αριθμός παιδιών). Αν ψάχνουμε για το τελευταίο ghost, τότε μετά καλούμε τη `max_value` γιατί παίζει ο παίκτης `max`(δηλαδή το `pacman`). Αν όμως δεν ψάχνουμε για το τελευταίο ghost, τότε καλούμε την `expectimax_value` για το επόμενο ghost(δηλαδή για το ghost με index μεγαλύτερο κατά 1 από το index του προηγούμενου ghost. Για κάθε κόμβο που δεν είναι φύλλο, υπολογίζουμε το μέσο όρο των τιμών όλων των κόμβων-παιδιών του και επιστρέφουμε αυτό. Για κάθε παιδί του state-ρίζα, που είναι το state που παράγεται με κάθε κίνηση `action` που μπορεί να κάνει το `pacman`, καλούμε την `expectimax_value` και βρίσκουμε την `expectimax` τιμή του. Έπειτα, βάζουμε τη πλειάδα (`expectimax_value, action`) σε μία λίστα. Επειδή ο κόμβος-ρίζα είναι `max`, επιστρέφουμε τη κίνηση που έχει τη μεγαλύτερη `expectimax` τιμή.

Ερώτημα 5:

Η συνάρτηση αξιολόγησης στον κώδικα επιστρέφει μία τιμή που είναι ίση με το άθροισμα του `score` του state και μίας τιμής `evaluation` η οποία υπολογίζεται με τον παρακάτω τρόπο: Αρχικά ελέγχει τα ghosts. Για κάθε ghost που δεν είναι τρομαγμένο υπολογίζει την απόστασή του από το `pacman`. Αν συναντηθούν ή βρεθούν σε διπλανές θέσεις τότε η `evaluation` μειώνεται διότι το καλύτερο state είναι αυτό με το μεγαλύτερο `evaluation`. Αν το ghost είναι τρομαγμένο, αυτό σημαίνει ότι το `pacman` μπορεί να το φάει. Άρα η `evaluation` αυξάνεται. Έπειτα ελέγχει τα φαγητά. Βρίσκει τη μικρότερη απόσταση ανάμεσα σε ένα `food` και στο `pacman` και προσθέτουμε

στην evaluation την τιμή $(5 - 0.5 * \text{απόσταση})$. Αυτό σημαίνει πως όσο πιο μακριά βρίσκεται το κοντινότερο φαγητό στο pacman από το pacman, τόσο μικρότερη είναι η αύξηση του evaluation (αν είναι μεγάλη η απόσταση μπορεί και να μειωθεί το evaluation).